



गृह मंत्रालय
MINISTRY OF
HOME AFFAIRS

राष्ट्रीय न्यायिक विज्ञान विश्वविद्यालय
National Forensic Sciences University



Essentials of Cyber Security and Cyber Warfare



Dr. Lokesh Chouhan
Associate Professor



गृह मंत्रालय
MINISTRY OF
HOME AFFAIRS

राष्ट्रीय न्यायालयिक विज्ञान विश्वविद्यालय
(राष्ट्रीय महत्त्व का संस्थान, गृह मंत्रालय, भारत सरकार)
National Forensic Sciences University
(An Institution of National Importance under Ministry of Home Affairs,
Government of India)



E-Mail: Lokeshchouhan@gmail.com, Lokesh.chouhan_goa@nfsu.ac.in

Mob: +91-898924399, 9827235155



Information Security CSD-410



Instructor: Dr. Lokesh Chouhan
Assistant Professor

Computer Science and Engineering (CSE) Department
National Institute of Technology (NIT)
Hamirpur (H.P.) INDIA

E-Mail: Lokeshchouhan@gmail.com, lokesh@nith.ac.in

Mob: +91-898924399, 9827235155





Unit 4

Public Key Cryptography

Private-Key Cryptography

- traditional **private/secret/single key** cryptography uses **one** key
- shared by both sender and receiver
- if this key is disclosed communications are compromised
- also is **symmetric**, parties are equal
- hence does not protect sender from receiver forging a message & claiming is sent by sender

Public-Key Cryptography

- probably most significant advance in the 3000 year history of cryptography
- uses **two keys** – a **public** & a **private key**
- **asymmetric** since parties are **not** equal
- uses clever application of number theoretic concepts to function
- complements **rather than** replaces private key crypto

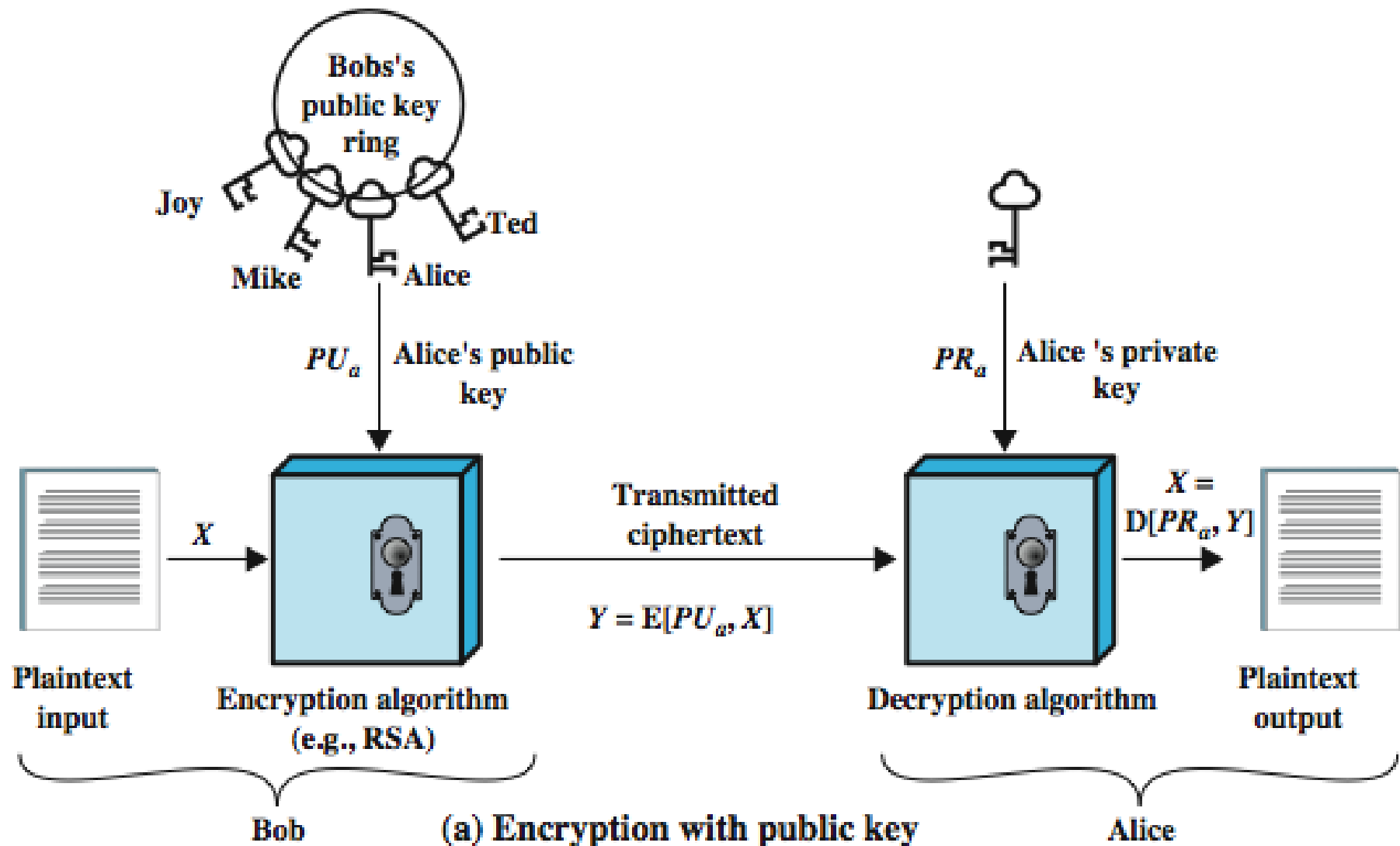
Why Public-Key Cryptography?

- developed to address two key issues:
 - **key distribution** – how to have secure communications in general without having to trust a KDC with your key
 - **digital signatures** – how to verify a message comes intact **from the claimed sender**
- public invention due to Whitfield **Diffie** & Martin **Hellman** at Stanford Uni in **1976**
 - known earlier in classified community

Public-Key Cryptography

- **public-key/two-key/asymmetric** cryptography involves the use of **two** keys:
 - a **public-key**, which **may be known by anybody**, and can be used to **encrypt messages**, and **verify signatures**
 - a related **private-key**, known **only to the recipient**, used to **decrypt messages**, and **sign (create) signatures**
- **infeasible to determine private key from public**
- is **asymmetric** because
 - those who encrypt messages or verify signatures **cannot** decrypt messages or create signatures

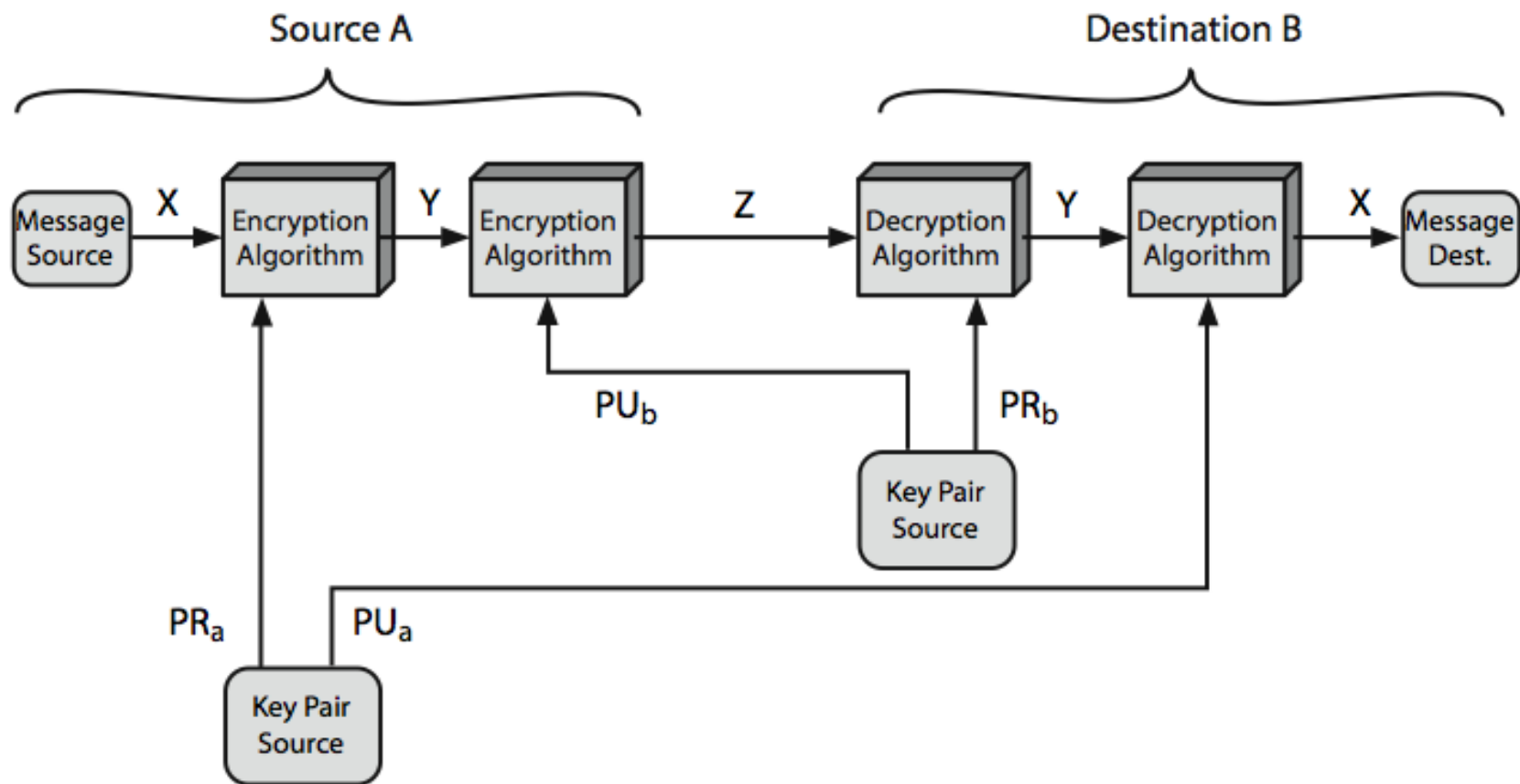
Public-Key Cryptography



Symmetric vs Public-Key

Conventional Encryption	Public-Key Encryption
<i>Needed to Work:</i> 1. The same algorithm with the same key is used for encryption and decryption. 2. The sender and receiver must share the algorithm and the key. <i>Needed for Security:</i> 1. The key must be kept secret. 2. It must be impossible or at least impractical to decipher a message if no other information is available. 3. Knowledge of the algorithm plus samples of ciphertext must be insufficient to determine the key.	<i>Needed to Work:</i> 1. One algorithm is used for encryption and decryption with a pair of keys, one for encryption and one for decryption. 2. The sender and receiver must each have one of the matched pair of keys (not the same one). <i>Needed for Security:</i> 1. One of the two keys must be kept secret. 2. It must be impossible or at least impractical to decipher a message if no other information is available. 3. Knowledge of the algorithm plus one of the keys plus samples of ciphertext must be insufficient to determine the other key.

Public-Key Cryptosystems



Public-Key Applications

- can classify uses into **3 categories**:
 - **encryption/decryption** (provide **secrecy**)
 - **digital signatures** (provide **authentication**)
 - **key exchange** (of session keys)
- some algorithms are suitable for all uses, others are specific to one

Algorithm	Encryption/Decryption	Digital Signature	Key Exchange
RSA	Yes	Yes	Yes
Elliptic Curve	Yes	Yes	Yes
Diffie-Hellman	No	No	Yes
DSS	No	Yes	No

Public-Key Requirements

- Public-Key algorithms rely on two keys where:
 - it is computationally **infeasible to find decryption key knowing only algorithm & encryption key**
 - it is computationally **easy to en/decrypt messages when the relevant (en/decrypt) key is known**
 - either of the two related **keys can be used for encryption, with the other used for decryption (for some algorithms)**
- these are formidable requirements which only a few algorithms have satisfied

One-way functions

- Most common functions are *invertible*; for any $F(x) = y$, there is an $F^{-1}(y) = x$.
 - Multiplication and division
 - DES
- A function which is easy to compute in one direction, but hard to compute in the other, is known as a *one-way function*.
 - Hashing, modular arithmetic.
- A one-way function that can be easily inverted with an additional piece of knowledge is called a *trapdoor one-way function*.

One-way functions

- Public key encryption is based on the existence of **trapdoor one-way functions**.
 - Encryption with the public key is easy.
 - Decryption is computationally hard.
 - Knowledge of the private key opens the trapdoor, making inversion easy.
- Password systems also use one-way functions.

Public-Key Requirements

- need a trapdoor one-way function
- one-way function has
 - $Y = f(X)$ easy
 - $X = f^{-1}(Y)$ infeasible
- a trap-door one-way function has
 - $Y = f_k(X)$ easy, if k and X are known
 - $X = f_k^{-1}(Y)$ easy, if k and Y are known
 - $X = f_k^{-1}(Y)$ infeasible, if Y known but k not known
- a practical public-key scheme depends on a suitable trap-door one-way function

Modular Arithmetic

- **RSA's security** is based on modular arithmetic.
 - $a = b \pmod{n} \Leftrightarrow$ there is a q such that $a-b=qn$
 - b is the remainder after dividing a by n
 - $23 = 3 \pmod{5}$
- A set $\{0,1,\dots,n-1\}$ is *closed* under modular addition and multiplication.
- $(a \pmod{n} + b \pmod{n}) \pmod{n} = (a+b) \pmod{n}$
- $(ab) \pmod{n} = (a \pmod{n} b \pmod{n}) \pmod{n}$

Identities and Inverses

- An identity is a number that maps a number to itself under some operation.
 - 0 in normal addition, 1 in multiplication.
- An inverse is a number (within the input set) and maps a given number to the identity
 - $X * 1/X$, $X + -X$ in integer math
- **We are particularly interested in multiplicative inverses for modular arithmetic.**
 - **$(ab) = 1 \pmod{n}$**

Multiplicative Inverses

- 3 and 2 are multiplicative inverses mod 5.
- 7 and 6 are multiplicative inverses mod 41.
- 5 and 2 are multiplicative inverses mod 9.
- For $n > 1$, if **a** and **n** are relatively prime, there is a unique x such that
 - **$ax = 1 \pmod{n}$**

More preliminaries

- **Fermat's Little Theorem:**
 - If p is prime, then for all a :
 - $a^{p-1} = 1 \pmod{p}$
- **Chinese Remainder Thm** (corollary)
 - If p and q are prime, then for all x and a :
 - $x = a \pmod{p}$ and $x = a \pmod{q}$ iff $x = a \pmod{pq}$
- These are needed to prove RSA's correctness.

RSA

- by **R**ivest, **S**hamir & **A**dleman of **MIT** in **1977**
- best known & widely used public-key scheme
- based on exponentiation in a **finite (Galois) field over integers modulo a prime**
 - nb. exponentiation takes $O((\log n)^3)$ operations (easy)
- uses large integers (eg. 1024 bits)
- security due to cost of **factoring large numbers**
 - nb. factorization takes $O(e^{\log n \log \log n})$ operations (hard)

The RSA Algorithm

- Pick two large (100 digit) primes p and q .
- Let $n = pq$
- Select a relatively small integer d that is prime to $(p-1)(q-1)$
- **Find e** , the multiplicative inverse of $d \bmod (p-1)(q-1)$
- (d,n) is the **public key**. To encrypt M , compute
 - $En(M) = M^e \bmod n = C$
- (e,n) is the **private key**. To decrypt C , compute
 - $De(C) = C^d \bmod n = M$

RSA En/decryption

- to encrypt a message M the sender:
 - obtains **public key** of recipient $PU = \{e, n\}$
 - computes: $C = M^e \bmod n$, where $0 \leq M < n$
- to decrypt the ciphertext C the owner:
 - uses their **private key** $PR = \{d, n\}$
 - computes: $M = C^d \bmod n$
- note that the message M must be smaller than the modulus n (block if needed)

Why RSA Works

- because of **Euler's Theorem**:
 - $a^{\phi(n)} \bmod n = 1$ where $\gcd(a, n) = 1$
- in RSA have:
 - $n = p \cdot q$
 - $\phi(n) = (p-1)(q-1)$
 - carefully chose **e** & **d** to be inverses **mod** $\phi(n)$
 - hence **e.d = 1 + k.φ(n)** for **some k**
- hence :
$$C^d = M^{e \cdot d} = M^{1+k \cdot \phi(n)} = M^1 \cdot (M^{\phi(n)})^k$$
$$= M^1 \cdot (1)^k = M^1 = M \bmod n$$

Correctness of RSA

- To show **RSA is correct**, we must show that encryption and decryption are inverse functions:
 - $\text{En}(\text{De}(M)) = \text{De}(\text{En}(M)) = M = M^{\text{ed}} \pmod{n}$
 - Since **d** and **e** are multiplicative inverses, there is a **k** such that:
 - $ed = 1 + kn = 1 + k(p-1)(q-1)$
 - $M^{\text{ed}} = M^{1+k(p-1)(q-1)} = M * (M^{p-1})^{k(q-1)}$
 - By **Fermat**: $M^{p-1} = 1 \pmod{p}$
 - $M^{\text{ed}} = M(1)^{k(q-1)} \pmod{p} = M \pmod{p}$

Correctness of RSA

- $M^{\text{ed}} = M(1)^{k(q-1)}(\text{mod } p) = M(\text{mod } p)$
- $M^{\text{ed}} = M(1)^{k(q-1)}(\text{mod } q) = M(\text{mod } q)$
- By **Chinese Remainder Thm**, we get:
- $M^{\text{ed}} = M (\text{mod } p) M (\text{mod } q) =$
 $M (\text{mod } pq) = M (\text{mod } n)$
- Therefore, **RSA reproduces the original message and is correct.**

RSA Example - Key Setup

1. Select primes: $p=17$ & $q=11$
2. Calculate $n = pq = 17 \times 11 = 187$
3. Calculate $\phi(n) = (p-1)(q-1) = 16 \times 10 = 160$
4. Select e : $\gcd(e, 160) = 1$; choose $e=7$
5. Determine d : $de = 1 \pmod{160}$ and $d < 160$
Value is $d=23$ since $23 \times 7 = 161 = 10 \times 160 + 1$
6. Publish **public key** $PU = \{7, 187\}$
7. Keep secret **private key** $PR = \{23, 187\}$

RSA Example - En/Decryption

- sample RSA encryption/decryption is:
- given message **M = 88** (nb. $88 < 187$)

- encryption:

$$C = 88^7 \bmod 187 = 11$$

- decryption:

$$M = 11^{23} \bmod 187 = 88$$

Question

- $p = 11$, $q = 13$
- $M=42$

Obtain:

- 1) Public Key
- 2) Private key
- 3) Cipher Text

RSA example 1

- Let $p = 11$, $q = 13$
- $n = pq = 143$
- $(p-1)(q-1) = 120 = 3 \times 2^3 \times 5$
- Possible d : 7, 11, 13, 17, ... (let's use 7)
- Find e : $e \cdot 7 = 1 \pmod{120} = 103$
- Public key: (7, 143)
- Private key: (103, 143)
- $En(42) = 42^7 \pmod{143} = 81$
- $De(81) = 81^{103} \pmod{143} = 42$

Key Generation

Select p, q	p, q both prime, $p \neq q$
Calculate $n = p \times q$	
Calculate $\phi(n) = (p-1) \times (q-1)$	
Select integer e	$\gcd(\phi(n), e) = 1; 1 < e < \phi(n)$
Calculate d	
Public key	$KU = \{e, n\}$
Private key	$KR = \{d, n\}$

Encryption

Plaintext:	$M < n$
Ciphertext:	$C = M^e \pmod{n}$

Decryption

Ciphertext:	C
Plaintext:	$M = C^d \pmod{n}$

Fig. 2. RSA Algorithm

Question

- $p = 61$, $q = 53$
- $M=4$

Obtain:

- 1) Public Key
- 2) Private key
- 3) Cipher Text

1. Choose two distinct prime numbers, such as
 $P=61$ and $Q=53$.

2. Compute $n = p * q$ giving:
 $n=61*53=3233$

3. Compute $\phi(n) = (p - 1)(q - 1)$ giving:
 $\phi(3233) = (61 - 1)(53 - 1) = 60 * 52 = 3120$

4. Choose any number $1 < e < 3120$ that is co-prime
to 3120
Let $e=17$

5. Compute a value for d such that
 $(d * e) \% \phi(n) = 1$. One solution is:
 $d=2753$

Generation of Public and Private Key
(In RSA)

RSA example 2

- Let $p = 11$, $q = 3$, $M=7$
- Find

Public, Private, ciphertext and Decihpertext?

RSA example 2

- Let $p = 11$, $q = 3$, $M=7$
- $n = pq = 33$
- $(p-1)(q-1) = 20$
- Possible d : 3, 7, 11, 13, 17, ... (let's use 7)
- Find e : $e * 3 = 1(\text{mod } 20) = 7$
- Public key: (3, 33)
- Private key: (7, 33)
- $En(7) = 7^3 (\text{mod } 33) = 13$
- $De(13) = 13^7 (\text{mod } 33) = 7$



RSA example 3

- Let **P = 53** and **Q = 59**,
- Now we will encrypt “**HI**” :



RSA example 3

- Convert letters to numbers :

H = 8 and I = 9

M=89

- Choose $e=3$



RSA example 3

- **Public Key (e,n) =**
Public Key (and $e = 3$, $n = 3127$) and
- **Private Key(d,n)**
Private Key($d = 2011$, 3127)
- **Cipher Text = $89^e \bmod 3127$**
 $C = 1394$
- **Decipher = $1394^d \bmod 3127$**
 $M = 89$

Exponentiation

- can use the **Square and Multiply Algorithm**
- a fast, efficient algorithm for exponentiation
- concept is based on **repeatedly squaring base**
- and multiplying in the ones that are needed to compute the result
- look at binary representation of exponent
- only takes **$O(\log_2 n)$ multiples** for **number n**
 - eg. $7^5 = 7^4 \cdot 7^1 = 3 \cdot 7 = 10 \pmod{11}$
 - eg. $3^{129} = 3^{128} \cdot 3^1 = 5 \cdot 3 = 4 \pmod{11}$

Exponentiation

```
c = 0; f = 1
for i = k downto 0
    do c = 2 x c
        f = (f x f) mod n
    if bi == 1 then
        c = c + 1
        f = (f x a) mod n
return f
```

Efficient Encryption

- encryption uses exponentiation to power e
- hence if e small, this will be faster
 - often choose $e=65537$ ($2^{16}-1$)
 - also see choices of $e=3$ or $e=17$
- but if e too small (eg $e=3$) can attack
 - using Chinese remainder theorem & 3 messages with different moduli
- if e fixed must ensure $\gcd(e, \phi(n)) = 1$
 - ie reject any p or q not relatively prime to e

Efficient Decryption

- decryption uses exponentiation to power d
 - this is likely large, insecure if not
- can use the Chinese Remainder Theorem (CRT) to compute mod p & q separately. then combine to get desired answer
 - approx 4 times faster than doing directly
- only owner of private key who knows values of p & q can use this technique

RSA Key Generation

- users of **RSA must**:
 - determine two primes at random p, q
 - select either **e** or **d** and compute the other
- primes p, q must **not be easily** derived from modulus $n=p \cdot q$
 - means must be **sufficiently large**
 - typically **guess and use probabilistic test**
- exponents **e, d** are inverses, so use Inverse algorithm to compute the other



Security of Public Key Schemes

- like private key schemes brute force **exhaustive search attack** is always theoretically possible
- but keys used are **too large (>512bits)**
- security relies on a **large enough** difference in difficulty between **easy** (en/decrypt) and **hard** (cryptanalyse) problems
- more generally the **hard** problem is known, but is made hard enough to be impractical to break
- requires the use of **very large numbers**
- hence is **slow** compared to **private key schemes**

RSA Security

- possible approaches to attacking RSA are:
 - **brute force** key search - infeasible given size of numbers
 - **mathematical attacks** - based on difficulty of computing $\phi(n)$, by factoring modulus n
 - **timing attacks** - on running of decryption
 - chosen ciphertext attacks - given properties of RSA

Factoring Problem

- mathematical approach takes 3 forms:
 - factor $n=p \cdot q$, hence compute $\phi(n)$ and then d
 - determine $\phi(n)$ directly and compute d
 - find d directly
- currently believe all equivalent to factoring
 - have seen slow improvements over the years
 - as of May-05 best is 200 decimal digits (663) bit with LS
 - biggest improvement comes from improved algorithm
 - cf QS to GHFS to LS
 - currently assume 1024-2048 bit RSA is secure
 - ensure p, q of similar size and matching other constraints

Strengths of RSA

- No prior communication needed
- Highly secure (for large enough keys)
- Well-understood
- Allows both encryption and signing

Weaknesses of RSA

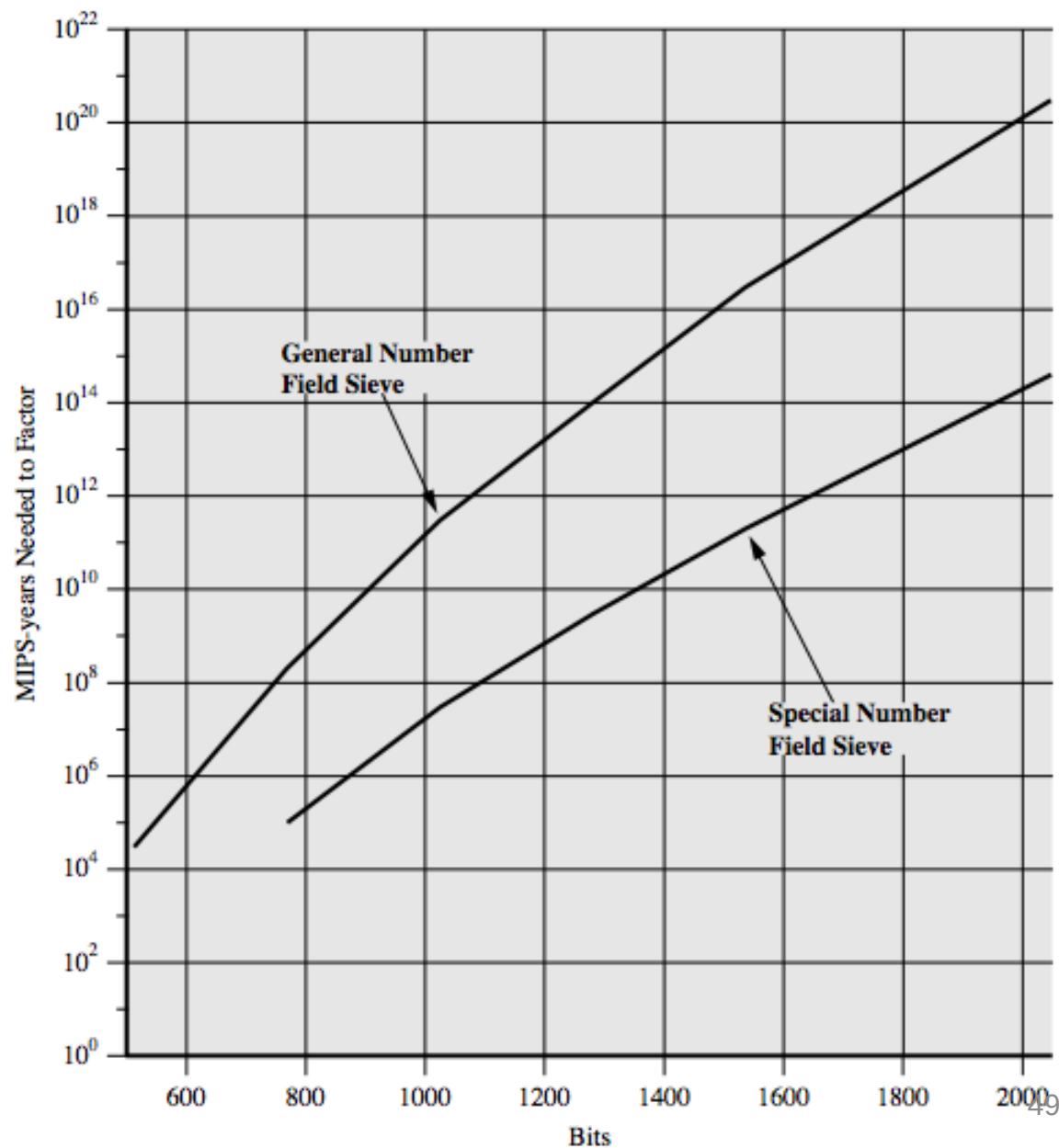
- Large keys needed (1024 bits is current standard)
- Relatively slow
 - Not suitable for very large messages
- Public keys must still be distributed safely.

Progress in Factoring

Number of Decimal Digits	Approximate Number of Bits	Date Achieved	MIPS-years	Algorithm
100	332	April 1991	7	quadratic sieve
110	365	April 1992	75	quadratic sieve
120	398	June 1993	830	quadratic sieve
129	428	April 1994	5000	quadratic sieve
130	431	April 1996	1000	generalized number field sieve
140	465	February 1999	2000	generalized number field sieve
155	512	August 1999	8000	generalized number field sieve
160	530	April 2003	—	Lattice sieve
174	576	December 2003	—	Lattice sieve
200	663	May 2005	—	Lattice sieve



Progress in Factoring



Timing Attacks

- developed by **Paul Kocher** in **mid-1990's**
- exploit **timing variations in operations**
 - eg. multiplying by small vs large number
 - or IF's varying which instructions executed
- infer operand size based on time taken
- RSA exploits time taken in exponentiation
- **countermeasures**
 - use constant exponentiation time
 - add random delays
 - blind values used in calculations

Chosen Ciphertext Attacks

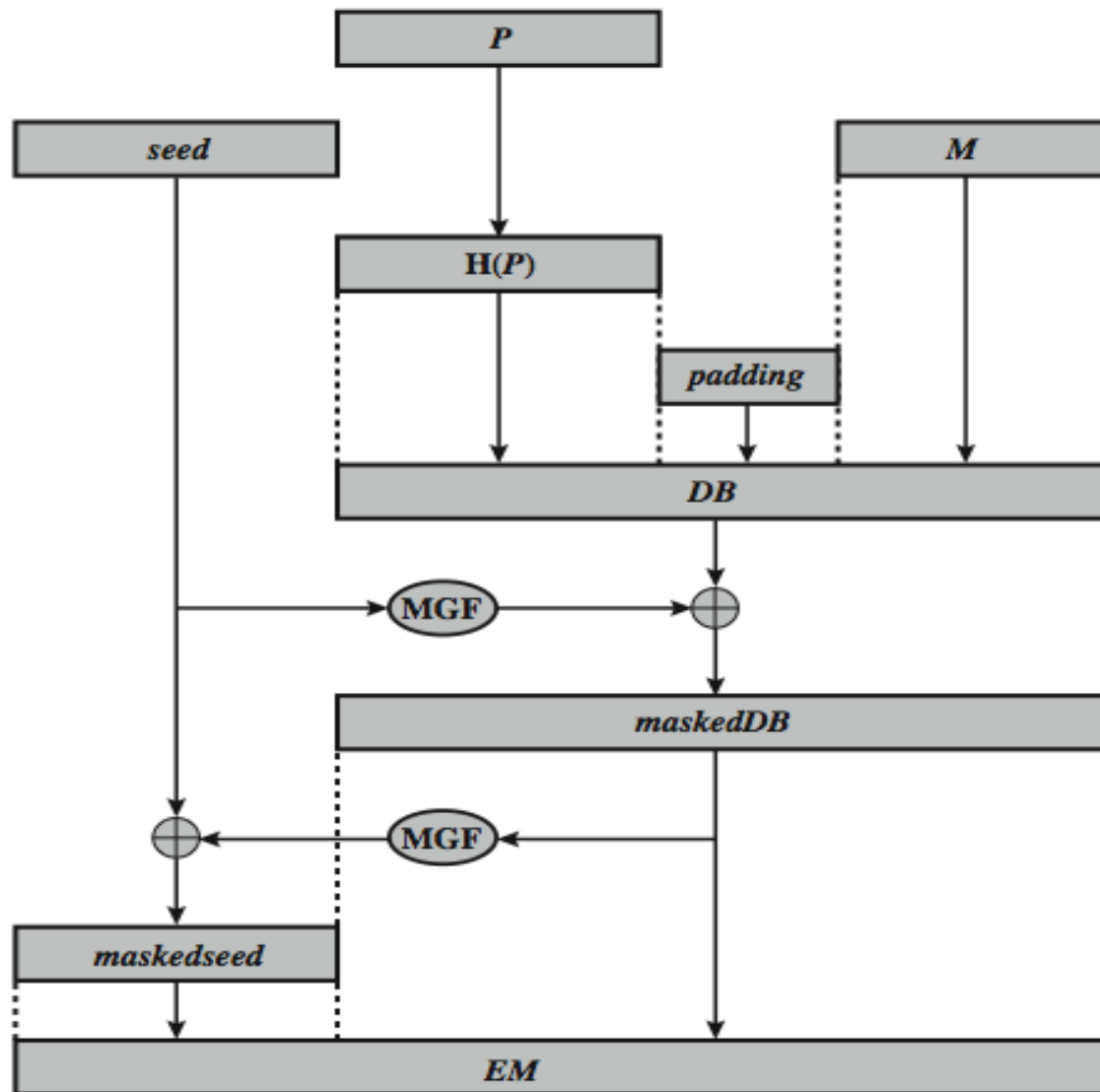
RSA is vulnerable to a Chosen Ciphertext Attack (CCA)

attackers chooses ciphertexts & gets decrypted plaintext back

choose ciphertext to exploit properties of RSA to provide info to help cryptanalysis

can counter with random pad of plaintext or use Optimal Asymmetric Encryption Padding (OASP)

Optimal Asymmetric Encryption Padding (OASP)



P = encoding parameters
 M = message to be encoded
 H = hash function

DB = data block
 MGF = mask generating function
 EM = encoded message

Summary

- have considered:
 - principles of public-key cryptography
 - RSA algorithm, implementation, security